Material for 8-12 year olds

Link to the book as it stands today: https://github.com/nagydani/compy-docs/tree/26d53bad7b53c7ebf2ef29d21ad8c5f731088f41

- A methodology that we can use in clubs, where we have ~10 kids, a the age of 8-12
  - incl lesson plan, or plans
- Self-study material
  - a book
  - or other formats
- Figure out some idea for Dani to do in a week or two

——-

not a goal

- **non-goal:** teach software engineering
  - **goal:** Dani: we want to teach wizardry
- **non-goal:** attempt to produce people who confirm to some standards - we are not producing homo sapiens for industrial use
  - **goal:** pass on the feeling that we know is possible to come from solving problems, the whole experience, and help kids build the self image that they are capable
- **non-goal:** create a researcher mindset
  - goal: enable kids to feel ...
- **goal:**
  - we want to provide them with additional avenues of self-expression

Principles

- do not train kids out from their curiosity
-

——-

Problem space discovery

-
- How to create and maintain the state of flow?
- Storytelling, incl. non-interactive demos, as self-expression.
- Stuff other than programming that can be learned with a computer much easier than without one.
- Competitive activities. (e.g. RedCode was cool — for a while, robot sumo is cool)

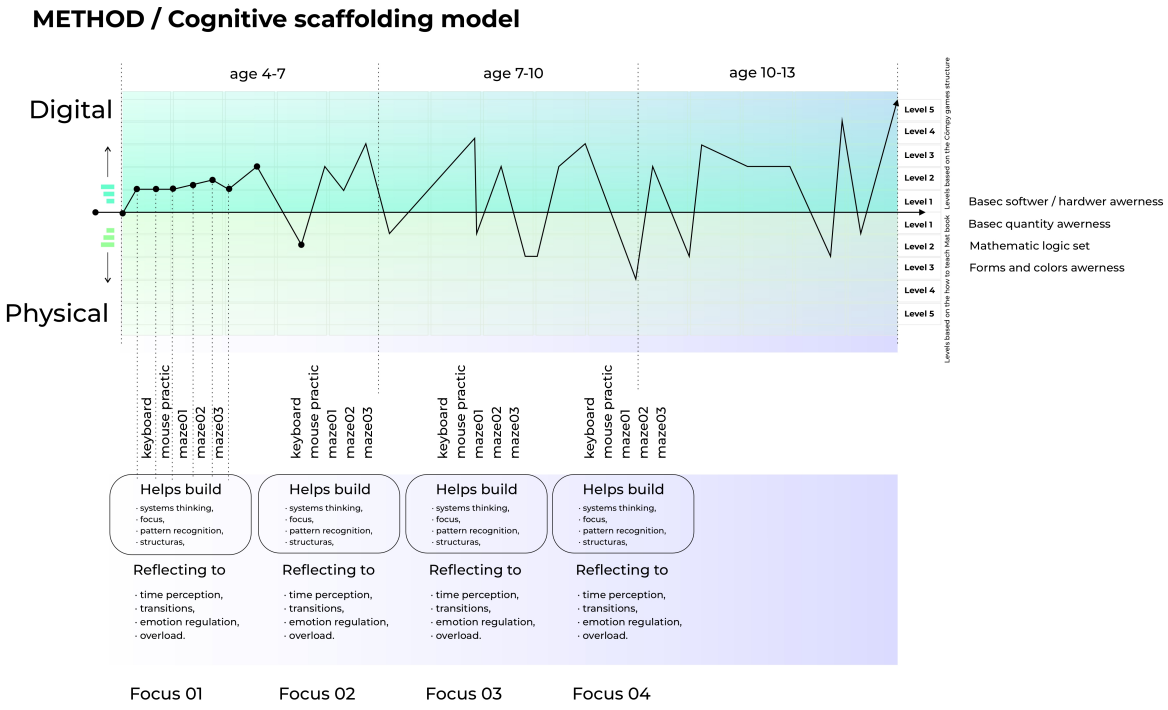
    - What other competitive activities we can think about in this classroom
        - One is red-code: try to take over some code, make championship, everybody submits their own, whose program is the champion
    - Robot-sumo: it can be done with identical robots programmed differently, two robots in an area
- Collaborative activities.
    - doing something together by splitting up the work..
    - Nándor: have experience: you pre-write the API for the video guy and the audio guy
        - at uni, for people who just started out to code.
- Procedural generation of stuff found in nature is cool (snowflakes, geography, starry skies, rocks, plants, water, fire etc.)
    - Enjoyed: Beginning with age 10 and later - programming stuff that resembles nature, e.g. generate things that look like real fireworks — there's so many things you can do, like fire, planet, earth-like planet, moon-like planet, small steps, iterate them, looks much more complex than what it really is.
        - Danila: programming competition: program that was a demo of seasons. It showed a landscape that changed seasons.
-
- Cool: colorful keyboard. Link: https://ws.baldvin.net/f/20290
- Cool: project gallery in Scratch
    - Not cool: most cool projects are overwhelmingly complex and just opening them demotivates some kids
        - Corollary: complexity ladder while staying cool on the way there (no boring "baby step" abstract examples—we must do something at least somewhat cool at every step)
- Cool: a lot of ready to use assets (sprites, animations, sounds, music) — enables focusing on quickly achievable stuff
    - Also very cool: an easy way to digitize own stuff (drawings, photos, videos etc.) and use them — for someone capable of doing that it's a big plus
- Very uncool: oracle-like experience (AI, to the lesser degree—books)
    - AI as a tutor giving hints can be cool
    - too trivial
    - how a lot of people use AI
    - a lot of kids use AI to solve homework, bad experience —- any kind of pre-made answer
        - experience: when I work, I think hard, then I can do
-
-
-
-
-
-

- It is a wrong goal wanting to create something which works without the parent or teacher ever pushing. Anecdote: mother of a violin player: “sure, sometimes she cries, but that’s normal”.
- The activity itself must be the reward.
- The Pólya method is pretty much the only way — everything has to be a question, and discovered together.
-
-
- Some people (kids and adults as well) cannot _possibly_ perform during interaction with others, and need alone time to try out something.
-
-
-
-
-
-
-
- Try not to keep the environment seem too “sheltering”; Although getting kids to code is cool enough, its very hard to escape py turtle, and notice that one can interact with the wider world
    - Dont hide rendering, IO, etc too much behind abstraction for simplicity?
    - does not give the feeling that one can do stuff
    - Dani: agree, I think that what we’re teaching that it’s the first steps towards what is done in real, not just a toy where you exhaust all possibilities, that’s it. Avoid the plastic.
        - Baldvin: example: transistor
        - **Danila: needed to work with**
- Avoid beeing a “Tutorial”; Leading kids to understand how to _solve_ the problems (and that they can solve them!) is a lot more important than actually having the problems solved
    - I asked people how to do something: I can show you, they never showed me how to figure it out. **They never explained how to figure it on your o**wn.
    - Show them how to figure out how to do stuff
    - Dani: I have a feeling that we are projecting our things to the world
        - Seems that some people like to do more and more
        - Dani: do 2-3, but then I want to plan on my own.
            - Katya: all of them reached that point, that not only they want to create their own rules but want to make their own universe, he tried to build some kind of rocket, and then some spacecrafts, and so on. He can do anything with this lego-box, but he decided not to, and after all these rockets and spacecrafts, said please bring here the broken TV, I will bring a screwdriver and let’s figure out what I can do. I liked to play with broken electronic stuff.

    - Absolutely hate HOWTO-style explanations, that gives commands what you can use, but without explaining why. I hated doing things that I didn't fully understand
-
-
-
- to offer children goals that can maintain their motivation through multiple phases, so that they can experience the joy of creation on an experiential level and at the same time master design thinking
-
- Kartoffel: you can implement some robot, and you can see how long your product lives
-
- developing such a complex system, but rather building on each other, "children are not connected according to age, but based on their knowledge level.
-
-
- Linda: I'd like something that I can use too.
-
-
-

quick example:

Visual MAP idea about our education fomula
Everyting is just example



Second exercise

Nándor

– (Dangerous / probably Pitfall full idea, which might or might not be feasible with a Book): Add support for skipping over topics, and some helpful annotation on how to go back and reread specific things; Maybe also think about, what someone who skipped a part of the Book, could be missing, without noticing, and remind them?

- Nándor: I struggled with: I knew 60 % already, jumping was very hard, because they used some material from the earlier sections. I didn't have patience to it, but skipping was not possible.
    - Danila: This is exactly where AI tutor would shine.

Dani

The coolest game we have is the 3d pong. I'd dig with the editor in and show how changing different parameters or code snippets changes the game.

- sizes of the bats and the puck
- colors
- stickiness/elasticity of collisions
- number of lines in the grid
- sound effects

I think the motivation part is still not there, I want to dig into the game's code with the editor, show that if we change things, the game changes —- just to show that here's this magic language, you can write anything, and can change.

- Danila: my cool-on-meter is high for this, fully approved.
- Nándor: Also agree.
    - I would find it fun if we wouldn't just change surface level parameters, but meaningfully add some feature.

Baldvin

Build 2 simple games.

- Bomber: city from blockhouses, plane flies left-to-right back and forth, always 1 level lower, space drops a slow bomb, need to clear all building before crashing to them https://itch.io/e/9542994/basic-10liner-published-blitz-by-daniel-a-nagy
-
- Simple bouncing ball with a horizontal bat, super simple, only 45 degrees, no goal

Have elements in the book in sections:

- Event handling
- Draw simple shapes
- Abstraction of functionality, like "test if we reached the wall"

- Dani: it is planned, not next week
- Nándor: It was a fun thing to do.
- Danila: this might be too complex, depending on what the kid already knows.
    - Baldvin: going from zero to one, these games are selected for simplicity of the whole thing

Danila

I'd definitely start with some graphics/audio stuff.

Then introduce identifiers by creating a program that asks for a parameter and draws/beeps depending on what you enter

Then ideally something interactive, with a function or a loop, getting input and doing stuff depending on the input. A symbol representing the player moves around the playing field.

Then try to add some permanence using the identifiers and understand that having a field larger than maybe 3x2 is too complex for that. Then introduce a table to store the playing field.

I'd avoid introducing too many terms, e.g. I'd skip "keyword," "literal," "boolean," "integer" etc.

Function and table is fine. Variable is fine ("identifier" is not that important for the lesson)

- Dani: I tried that the last time. I showed him a few graphics commands, how to draw a line, how to draw a circle, how to change the color, some commands, he kind of enjoyed it, but he had this creative crisis, he was just sitting there, and didn't have an idea what to do with it, but he didn't
    - Nándor: when I was teaching children how to code, in such a situation what I did: I myself would start doing stuff, on my screen, doing fun stuff, sometimes they ask, sometimes they try to emulate
    - Danila: As someone who started like this, I didn't have my ideas for quite a while, I saw concrete things to draw, and then I ... I started with pre-made ideas, 6-7 pile of x,
        - 67 is a 4chen meme, where they try to popularize with random things, purpose: does not make sense, GNU of GenZ
- rest: increase complexity to then make it interactive —- thought about moving around the paying field
- string, concatenation, integer, boolean —- skip all that, leave function and variable, not even a distinction between a variable

Linda / it was redy I din't make in 12minutes =)

Balloon Game Ideas

It would be interesting if the games were connected to each other. In the Balloon Game, players could collect something that they could later use in another game.

Ideas

- Collecting pieces of a puzzle
- Collecting construction elements, bricks

- Medals and trophies feel somewhat meaningless — it would be much more valuable to collect objects that later become useful, functional, and meaningful.

01 — Construction Elements

Instead of simply giving points, the game could:

open new perspectives and “enrich your world.”

Players could collect construction elements that could later be used for building on the next level or in a future game.

The usefulness and complexity of the collected elements could relate to the difficulty of the completed tasks.

The colors of the construction elements could correspond to the colors of the tasks themselves.

These collected elements could appear in a small interface area:

- preferably in the upper left or right corner,
- since the lower part of the screen is more important for typing and interaction.

At the end of the game, a short visual summary could present the collected construction elements in a larger format so they are clearly visible.

Later, by completing other games, players could unlock blueprints for building:

- characters,
- robots,
- vehicles,
- or buildings.

At the beginning, these systems would still use:

- 2D elements,
- simple geometric shapes,
- and flat construction logic.

I also considered allowing children to collect LEGO elements, but perhaps that would be better introduced later, once they already have some foundational understanding of:

- 3D spatial perception,
- perspective,
- and digital construction.

From a neurological and developmental perspective, physical LEGO building and digital 3D environments require very different types of sensory processing and spatial cognition.
(Developmental Theory)

Cognitive Scaffolding Model

These games could function as the introductory stages of a larger learning sequence that gradually supports the development of design-oriented and spatial thinking skills.

It would be valuable to create a connected system between the games that:

- supports gradual progression,
- is sensorily understandable,
- manipulable,
- playful,
- and gently transitions children from physical space into abstract digital space.

The goal would be:

a gradual transition from concrete visual thinking toward abstract spatial representation.

Possible developmental stages:

- planes and simple geometric elements
- shapes and visual patterns
- connections, possibilities, shadow and depth
- isometric space
- perspective and full 3D thinking

This progression could help children slowly build:

- spatial reasoning,
- digital construction skills,
- systems thinking,
- and confidence in navigating abstract digital environments.

Dani: will sketch out the 45 degree ball

Danila: sketch out the drawing idea, have enough space for variety so the kid can change and its's not very rigid.

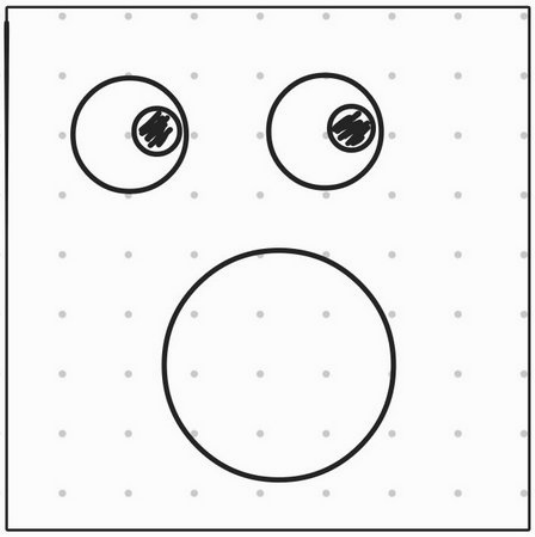
Baldvin: Bomber

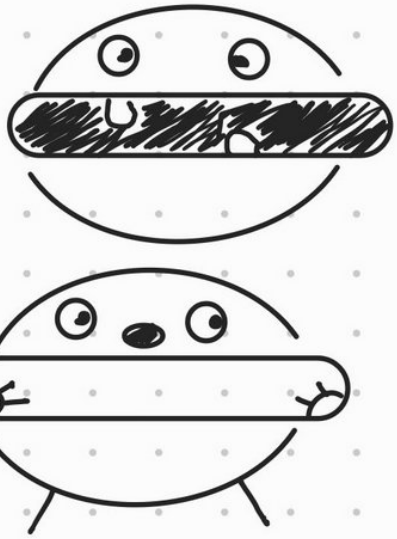
Danila's proposal

- Drawing a pair of eyes makes something immediately funny
- For this to inspire creativity we should start with something like this
- Showing a few sheets of drawings
- Depending on the audiencek
- We should have a pre-generated program drawing these, start tinkering with it right away - move eyes
- Change cute burger into sad burger by just replacing a few elements
- Small modifications of the program can change the picture a lot
- In response to pressing a key, it can change
- Filter: pack of fries is drawn in a primitive way, applying on the top makes it interesting
- Drawing/moving/permanent field stored in a table

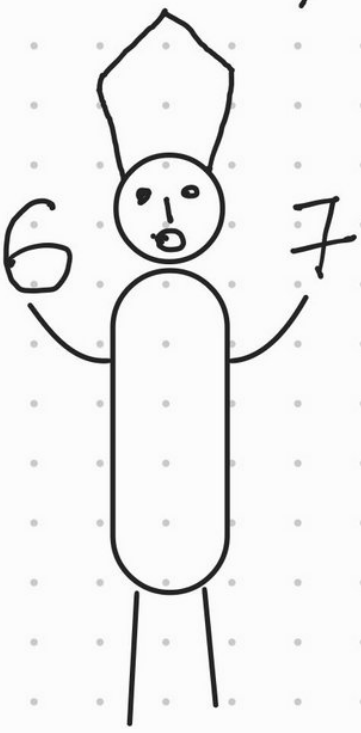
- Dani: I would totally do it in the first session
    - I would take 15 minutes preparation just to make people familiar with the command line, how to draw circles, squares, arcs
    - I also think, though maybe Linda can advis
    - the easiest technique for changing the size, drawing it on a grid
    - Make a grid on the wall and copy it square by square
    - You draw thing on a grid of dots, that gives you the coordinates
        - So it teaches subtle things
        - Euclidian coordinate space
        - Sequential execution
        - Instant gratification
        - I wouldn't use the result of this, I would take it as a complete circle, next time when we learn functions, I would start from scratch
- Nándi: I would try to make things much simpler to draw.
- Danila on tixy: there is a gap here that will require a lot of discretion. After showing something like tixy, we'd need a good explanation what function is, maybe from the hungarian methodology, maybe something else.
    - Specifically trigonometry can be hard to wrap your head around
- Dani: function is a topic that will need to be revisited about 10 time from the very beginning
    - Baldvin MONAD!!!!!!
- Dani: I had a beef with trigonometry. My dad explained it with the unit circle.

- Before I had access to computers before, putting a pixel at x and y coordinates was hard. A circle needs knowing sin and cos. So later on I never struggled with it.
- What I discovered early on: you almost never need trigonometry. Nice programs that do similar things, they can get away with using the Pythagorean theorem, or when you make harmonic motions, what you can do is you create a harmonic oscillator when the acceleration always points to the equilibrium, and the acceleration is proportional with the displacement. In practice it's almost always sufficient.
- Nándi: I don't like to teach trigonometry just to drawing a dot, I would propose skipping it using tricks you mention. However, it will still come up. Quite early on, children can have ideas which are intuitively solvable by trigonometry.
- Baldvin: humanity is not ready to drop it —- I'll work out some exercises
- Dani: other topic that can be learned very early. Systems of linear equations. I would approach it through graphics. How do you formulate —- intuitive notion of winding, a curve winds clockwise or counterclockwise - A, B, C - order clockwise or counterclockwise, but how do I figure out which is which. Intermediate step: some obvious transformations: mirror image symmetry, central point symmetry.
- After you learned to draw, there you learned there's a connection between arithmetics and coordinates, you can translate these things to numbers. Here's three points, what does it mean they are winding clockwise or counterclockwise. Here's two sections, do they intersect or don't they? At some point you can show that this thing which —- the determinant of a 2x2 matrix —- is a super powerful thing, it solves linear equations, tells if two things intersect or not, things are winding clockwise or counterclockwise —- when you go back and forth between coordinate system and linear
- Baldvin explained the approach of himself and his father: Inductive approach >>> deductive approach. Not knowing the formula is not a problem. But if you know a formula but can't draw a picture explaining why this formula works, you know nothing and this is unacceptable.
- Nandor: you don't teach determinant with the formula of determinant. You show it visually








6
7





Baldvin's

- Dani: I do like it for a second lesson.
    - For the previous iteration when I wanted to do it on the basis of the ZX spectrum, it was my second example as well
    - This is the kind of question typically you ask: don't you think this is going to be attacked by wokeists
- Nándi:
    - First sight it looked like the classic mistake of dropping long code in front of the child. 80% is incantation. Now that I'm reading through it it's a lot better.
    - I like how it is structured here.
    - Between the skeleton and the step-by-step construction guide people can spend 2 hours
- Dani: what do you think about giving good assets and high level things for the kids to play with. For example: having Linda draw nice characters, and being able to put nice characters on the screen, and animate those, with very little effort
- I thought that it will be a turn-off for many children, turns out not to be the case. Most children who play with Roblox, they genuinely believe that the game what they created is theirs
    - Nándi: it much depends on implementation
    - If you do it like scratch, or the usual scratch derivations, it is horrible
    - We need to investigate how Roblox achieves the feeling that one feels that it's mine
    - Dani: I think that Roblox is bad —- you are constrained, you don't have to be
        - Ininitally we would play with Linda's characters, but eventually we'd play with our own
        - Most importantly, the way currently we work, Linda draws SVGs, we have a transpiler, that transpiles the SVG code to LUA.
- Danila: on the "feeling that something is mine"
    - I'm not sure where you got the idea that kids feel this and this comes from
    - You can take this argument to the extreme,
    - The feeling that something is not mine comes from stupid unwarranted limitations - something that feels obvious that you can do it, but you cannot, it is forbidden by the system. E.g. when you could create your own sprites, but you are forced to use pre-made ones.
    - Children don't feel limited, or don't feel that something is not theirs, when they play with lego, just because they don't have triangular bricks.
- Dani: the technical risk of betting our course is too high, I'm not willing to go that far. But so that you know, with Vadim we are creating a 3D engine where the primitives are Lego stuff. It's a compy based projects, Compy, LUA, gives you a 3D gaming engine, instead of using triangles, quadri..., the primitives are Legos. It doesn't look worse than Roblox, it looks better, in Roblox your building blocks are bricks (orthogonally stretched cubes). Nice thing about LEGO is that you can take it in your hand and try things out. Lego: I expect fewer children bounce off it. Even if we don't do that: what do you think in the first

lesson we involve some pre-made higher quality assets, instead of using lines and points, we use the robby figures drawn by linda, flowers, trees, clouds.
- Nándor: very quickly build a trivial thing from 3 shapes, after show a pre-made asset of Linda, and have a peek inside it, see, it is the same thing in it
    - Does the code look bad.
- Dani: most robotics classes: because they are using the wrong abstractions, it's the way how I move, not how Baldvin moves while dancing. Go maximum acceleration, then they stop hard, instead of minimal energy. The other observation, because of this, some things that actual real life robots do all the time, these don't seem being even close. One thing that we should use primarily as a marketing tool: to build a unicycle robot.

Next steps:

- By next week, fold in the feedback and come up with a finalized MVP for the first 3 sessions.
- Next week we take a look at it, if we are comfortable, we accept them, and then we start a new round of brainstrorming for the remaining 5 sessins.

2026-06-23

Baldvin: Nándor, situation, you have 10 kids in a club. You choose your methodology. What would you do, and what materials you need?

Nándor:

- Show them that something that feels complicated is not complicated
- Try to look at their current stage, where they feel like they are restricted, or what is complicated
    - I can't code, I don't know where to write text
    - I can't write my game, because I don't really have ideas
        - Then you can show them how I would get ideas, how I try to find project ideas
        - If I don't have an idea how to do 3d graphics, I would be less inclined to give them a 3d graphics library, first I would show how to think
- Show them a way how to approach stuff
- Haven't used any toys for it:
    - I like a blackboard and my laptop, that's usually enough
    - I have never really done lessons that are pre-planned
    - I can imagine more walking in there, talking with them, why are you here, and then just starting something there.
    - Have a few things in hand already
        - working install of the Compy

        - after that if I show bomber, or SVG, or therminal: that would be pretty free
        - I would't go with the idea "today I'm gonna code bomber"
- Danila: I tried talk with one of the kids about ideas for new games
    - 9 year old kid
    - Out of 3 ideas he came up with, 3 were remixes or intermixes of the already existing Compy games
- Baldvin: Nándor's ideas about in a short synthesis:
    - We need to design our teacher instructions and lesson methodology such that initially a personal relationship building happens, and establishment of where we are, and setting local goals.
- Baldvin: Nándor: how can you implement the Pólya principles?
- Nándor:
    - I facilitate interaction as much as possible
    - depending on what we are doing, I try to sense what they already understand, and even more, they have a feeling they understand something, and I try to sense,where they have misunderstandings, and I try to find tasks that highlight those misunderstandings
    - alone, pairs, groups
- Baldvin: what if they are too much afraid of new things?
- Nándor:
    - Step back into their comfort zone
    - Let them do something
    - Not possible to find a single pattern
    - Someone may not be able to sit for 10 seconds
    - Some may be completely uncomfortable

- Situation: 5 of 8 y.o. Nándi and 5 of 8 y.o. Lóri forms the 10 people group
    - Suppose it is the first one, which ones are the Lóris and which ones are the Nándis
        - 5 of them are standing on the chair, 5 of them are sitting on the chair
    - You walk into the room
    - You have in your mind that you will go towards Bomber (or another starting project idea)
    - You talk with the kids — ask two
    - You have already an impression how viable it is to code bomber, one already did it, one never did anything
    - You try to find something to start out with, which is more or less good for most —- most of them already have done it, but are not confident with it - you tell them some checkpoint: start up the Compy, and draw a triangle
    - You give them not enough time to finish
        - If you give them enough time to finish, there will be a bunch who are just sitting there
    - You start working on it too, and tell them what you are currently doing.

    - Only those need to listen who wants to
    - Some feels like they are racing you
    - The ones who are afraid, they can code with you
    - Soon you will see who are copying you, and who are not
    - Pick out one who copy with you: ask them how should we proceed
    - If they answer, great, if they don't, no problem.
    - Iterate with this.
    - 15-20 minutes
    - Give them a head-start of 3-4 minutes
- Goal: after 45 minutes, everyone knows how to open the editor, and write a few lines of code, how to execute, and maybe how to troubleshoot. AND after this first exercise, the teacher knows who are the experimenting, extrovert, adhd kinds, and which are the more fearful ones, who maybe perform significantly under their full potential in unknown situations.

- Danila:
    - they are on different levels
    - main question relates to our conversation yesterday
    - there is a hard problem of interaction with a bunch of children and building trust with them, building good enough relationships so that they want to co-operate, they want to follow
    - this is a tricky business; no amount of instructions can fully solve this, while it can help
    - question: how do we create some kind of a requirement list to the person who is doing the thing
    - what the person should know and be able to do themselves before trying to teach kids
    - the random friend might be capable programming, understanding the platform.
    - there should be requirements against the teacher - we need to understand the persona who teaches
    - maybe provide some material to that person
    - so they train, even if they don't have required experience, they can train and prepare themselves
    - this prompts another thought: how the hungarian methodology solves that. A lot of teachers with various levels of knowledge and experience and motivation, and it tried to prepare all of them enough to be able to teach with that methodology. How was this tackled.
    - We can look into teacher's handbook of petterson's methodology. They tackle a lot of those questions. They have a bunch of ready to use recipes.
    - Random note: idea of showing kids something you do yourself, **we need to add a projecting device or a big-tv to stream from compy screen.** Without bringing too much attention to the fact that someone doesn't understand something. Showing to everyone: they can look or not look.

    -
- Nándor:
    - I like a lot how we do it in the olympiad trainings.
    - There are multiple teachers. One giving the lectures and the tasks.
    - The others actively are searching for issues, and helping 1:1.
    - Those lectures can be efficient.
    - Having 2 teachers for 20 is better than having 1 for 10.
- Baldvin & Danila:
    - Already we should have a teacher's manual, even if just for ourselves, that describes the expected and required behaviors from the teacher

We can assume that there is at least 2 teachers, so the dynamics mentioned above can be executed.

Now, what should happen after the short break:

For the first one, we'd continue similarly. However, it is important not to do this forever, because this does not lead to much learning. This is what you see in most courses, but they don't have an idea how to do something with kids that needs effort. Lot of kids don't do that just for fun. There has to come a time when it is a bit uncomfortable. Focusing on everyone being comfortable may result in depriving them from experiencing the reward of effort.

After the very first 45 minutes, so many of them are cooked, some lightweight exercise should be available, in a way that each can choose, maybe just playing for the rest of time that day, or choose an exercise from the paper handout, but no common track from here.

The second 45 minutes is lightweight, but we don't expose that, we cover a lot less. If you tell that "now you can relax", they will get bored. If they get bored, it will feel strenous after the fact, worse as if you guide a lesson, just slowly.

How do you create exercises to solve by them in this second half?

For "we code bomber together": so we are at this step, now it would be cool if the plane would fly, and then you tell them: okay, now you should make the bomb drop.

When you are in the second half, and you have 2 teachers, the second teacher can walk around, for bored ones, they can tell: Okay, bomber is not fun, have you seen this 3d pong, make arena red for me. A hard task - the issue was that they wanted to do stuff, ... etc.

We should have them re-develop the Knuth book, and do math. Even some lessons should be pen-and-paper lessons, theory lessons. I would structure it into:

1. continuing working on a project
2. math

3. info theory

Have different guides. You can vary what kind of session it is. After a lot of theory, it is improtant to do fun stuff too.

Theory lessons are problem solving lessons.

Do something on proofs.

Using Minesweeper as intro to proofs - not on the scond half of the first lesson, but on the second lesson.

Second lesson starts with minesweeper and proofs.

Danila:

- reinventing Knuth's book is a good goal, but working on something on their own is also an experience we want to have
- working on some project, working on something they will be able to show off with
- not just some solution of the problem that is virtually identical
- if everyone gets it right, the soution will be very similar.
- the experience building something on your own is good: maybe dedicate the second parts of the lesson to that
- other pattern: you choose a project, and you work on that project during many lessons. This has some problems. Main problem: those projects can diverge pretty seriously. It will be much harder to explain the same concepts to children who are working on different projects that diverged. One approach: a difference between a vertical and a horizontal tv series
    - some develop from episode to episode, end of an episode is the beginning of the next one
    - some have independent episodes
- One of the things we can do: the latter approach. Each lesson starts with a template, and maybe some particular task, some boilerplat code pre-written, they work with that, change that to solve a particular problem. We can choose the tasks in a way to allow them in the free time to further develop the thing into a more presonalized one.
- We can give them some ideas to play with.

There is a family of courses: Learn X the hard way. You don't just need to build a conceptual understanding, you need to build a muscle memory and pattern recognition. These courses insist that you type every program key by key, never copy-paste. Catch: our target audience might be completely non-familiar with typing on a keyboard. Typing even a small program, could easily take 30 minutes —- this will be a hard limit here. We won't be able to teach them to type quickly during those lessons. We need to resolve this, e.g. by having more boilerplate code pre-written. There should not be knowledge requirement for kids, we should accommodate.

Some synthesis:

- Lessons start with hard-core material, and second half is more relaxed
- First half is more constrained to the subject, second half is there for going to individual directions
- First lesson, first half is: editor and draw something, establish the personality and the knowledge level of each participant
- First lesson, second half: something very loose, but providing challenges for those who want it, letting just play those who don't
- Second lesson: starts with minesweeper and logic basics
- Overall: every lesson needs to have some underlying concepts (elements of computer science) which gets developed through exercise

https://git.baldvin.net/educationmatters/compy-project/src/branch/dev/dev/tools/build